

İskenderun Teknik Üniversitesi

Bilgisayar Mühendisliği

2025-2026 Bahar Dönemi

Mayıs 2026

Mühendislikte Bilgisayar Uygulamaları II

Dönem Projesi Raporu

Öğretim Üyesi: Halil İbrahim Okur

cmyLLMz

Yahşi Batı Filmi Mizah Analizi Sistemi

GitHub Deposu:

<https://github.com/ogzhntutucu/cmyLLMz>

Oğuzhan Tutucu (212523033)

1. Projenin Amacı

Bu projenin amacı, Cem Yılmaz'ın yazıp yönettiği "Yahşi Batı" (2010) filmindeki mizahı analiz eden, açıklayan ve sorgulanabilir hâle getiren bir **Retrieval-Augmented Generation (RAG)** tabanlı yapay zekâ sistemi geliştirmektir. Sistem, kullanıcının doğal dilde sorduğu sorulara filmin içeriğinden çıkarılan sahnelere dayanarak Türkçe cevap üretir.

Tipik kullanım senaryoları şunlardır:

- Bir şakanın neden komik olduğunu açıklamak ("Garfield şakası neden komik?")
- Bir karakterin mizah tarzını analiz etmek ("Lemi Galip ne tür mizah kullanıyor?")
- Belirli bir replik veya zaman damgasına ulaşmak ("Filmin 47. dakikasında kim ne diyor?")
- Mizah tekniğine göre sahneleri listelemek ("Filmde anakronizm kullanılan sahneler hangileri?")
- Kültürel bir referansı bağlamıyla açıklamak ("Teşkilât-ı Mahsusa nedir, filmde neden geçiyor?")

Sistem, filmin diyalog ve sahne yapısını manuel olarak işlenmiş ve zenginleştirilmiş bir veri setine dönüştürür; ardından bu veri setini bir vektör veri tabanına yükleyerek anlamsal arama (semantic search) ile soruya en alakalı sahneleri getirir ve büyük dil modeli (LLM) yardımıyla bağlamsal cevap üretir. Sonuç olarak kullanıcı, hem klasik bir arama motorundan, hem de bağlamsız bir sohbet botundan elde edemeyeceği şekilde **kaynaklı, açıklayıcı ve domain-specific** cevaplar alır.



cmyLLMz Streamlit Arayüzü

2. Problem Tanımı

Büyük dil modelleri (LLM) genel kültür sorularını başarıyla yanıtlayabilse de, dar alanlı (domain-specific) içeriklere — özellikle popüler olmayan veya niş bir Türk filminin sahne detaylarına — hâkim değildir. Doğrudan ChatGPT'ye “Yahşi Batı filminde Garfield şakası neden komik?” diye sorulduğunda, model genellikle:

- Sahneyi hatırlamadığı için **uydurma (hallucination)** yapar,
- “20-30. dakika civarı olabilir” gibi belirsiz tahminler verir,
- Mizahın işleyiş mekanizmasını yüzeyde geçer,
- Karakter isimlerini veya kültürel referansları karıştırır.

Çözmek istenen problem: Bir filmin diyaloglarını, sahnelerini, karakterlerini ve kültürel referanslarını yapılandırılmış bir veri setine dönüştürerek, LLM'in bu veriye **bağlı** cevap vermesini sağlamak; böylece hem hallucination'ı azaltmak, hem de cevap kalitesini (doğruluk + detay + tutarlılık) artırmak.

Daha genel formüle edilirse: **Yapılandırılmamış bir görsel-işitsel içeriği (film), kullanıcıyla doğal dilde diyaloga girebilen bir bilgi tabanına nasıl dönüştürürüz?** Bu projenin teknik cevabı, manuel olarak hazırlanmış yüksek kaliteli bir chunk veri seti + LLM zenginleştirme + bge-m3 embedding + ChromaDB + OpenAI gpt-5.4-mini pipeline'ıdır.

3. Veri Seti

3.1 Veri Kaynağı

Projenin tek hammaddesi, filmin Türkçe altyazı dosyasıdır (yahsi_bati.srt). Altyazı dosyası filmin tüm diyaloglarını zaman damgalarıyla (HH:MM:SS,mmm) içerir ancak şu eksiklikleri vardır:

- Konuşanın kim olduğunu söylemez (attribution yoktur)
- Görsel mizah (jest, mimik, sahne nesneleri) hakkında bilgi içermez
- Sahne sınırlarını belirtmez (tek bir akış olarak gelir)
- Kültürel referans veya mizah mekanizmasına dair açıklama içermez

Bu eksiklikleri kapatmak için altyazı dosyası, projenin omurgasını oluşturan **block_referans.md** dosyasına dönüştürülmüş ve **üç ayrı izleme** sırasında elle zenginleştirilmiştir.

3.2 Veri Hazırlama Yöntemi

Veri hazırlama, beş aşamalı bir süreçle yürütüldü:

Aşama 1 — SRT Parse: srt_parser.py ile altyazı dosyası parse edilip her bir altyazı birimi “block” olarak numaralandırıldı (toplam 1653 ham block, 9 tanesi tekrar/jenerik olduğu için silindi, 1644 anlamlı block).

Aşama 2 — block_referans.md Üretimi: Her satır NNNN [HH:MM:SS] altyazı metni formatında, zaman boşlukları --- ile ayrılmış olarak yazıldı.

Aşama 3 — 1. İzleme (Karakter Kodlaması + İlk Chunk Sınırları): Film baştan sona izlenerek her replikten önce konuşan karakterin iki harfli kodu (-AZ-, -LE-, -ZE-...) eklendi. 55 karakterin tamamı için kod=isim eşlemesi karakterler.txt'ye yazıldı. Anlamsal bütünlük taşıyan sahneler ===NNN=== işaretiyle ilk kez chunk'lara bölündü (50 chunk).

Aşama 4 — 2. İzleme (Detaylı İçerik Notları): Her sahnenin içine [köşeli parantez içinde] inline notlar yazıldı: kültürel referanslar, görsel gag açıklamaları, kelime oyunu mekanizmaları, gizli detaylar, anakronizm tespitleri. Chunk sınırları yeniden gözden geçirildi; uzun olanlar bölündü, kısa olanlar birleştirildi (sonuç: 62 chunk).

Aşama 5 — 3. İzleme (Lokasyon + Cross-Reference): Her chunk için @lokasyon X direktifi eklendi (62 direktif). Sahneler arası bağlantılar (tekrar eden esprileri, callback'leri, setup-payoff ilişkilerini yakalamak için) (bkz. yb_NNN) formatında işaretlendi (56 referans).

Bunlara ek olarak, izleme sırasında ASCII Türkçe ile yazılan inline notlar **GPT-4o-mini** ile diakritikli Türkçeye çevrildi (1107 inline not segmenti, ~\$0.01 maliyet). Yapısal elementler (block numarası, zaman damgası, karakter kodu, chunk sınırı) korundu.

3.3 Veri Şeması ve Örnek Veriler

Manuel hazırlanan block_referans.md dosyası, otomatik bir pipeline (chunk_merger.py + enricher.py) ile JSON şemasına dönüştürüldü. Üretilen enriched_chunks.json dosyasındaki tek bir chunk örneği aşağıdaki gibidir:

```
{
  "id": "yb_001",
  "start": "00:00:14",
  "end": "00:00:49",
  "duration_sec": 35.0,
  "block_range": "0001-0012",
  "text": "[Yahşi Batı filmi başlıyor. Dört karakter meyhanede...] -AL- Şimdi kadın...",
  "characters": {"AL": "Alpay", "RA": "Ramazan", "VE": "Vedat", "ZE": "Zeki"},
  "location": "beykoz meyhanesi",
  "related_chunks": ["yb_005", "yb_012"],
  "humor_analysis": {
    "techniques": ["anekdot", "wordplay"],
    "mechanism": "Bir erotik shop hikayesinin anekdot formunda anlatımı...",
    "cultural_context": "Türk meyhane kültürü ve hikâye anlatma geleneği..."
  },
  "summary": "Beykoz'da bir meyhanede dört arkadaş rakı masasında..."
}
```

Tablo 1, hazırlanan chunk veri setinden örnek bir parça göstermektedir:

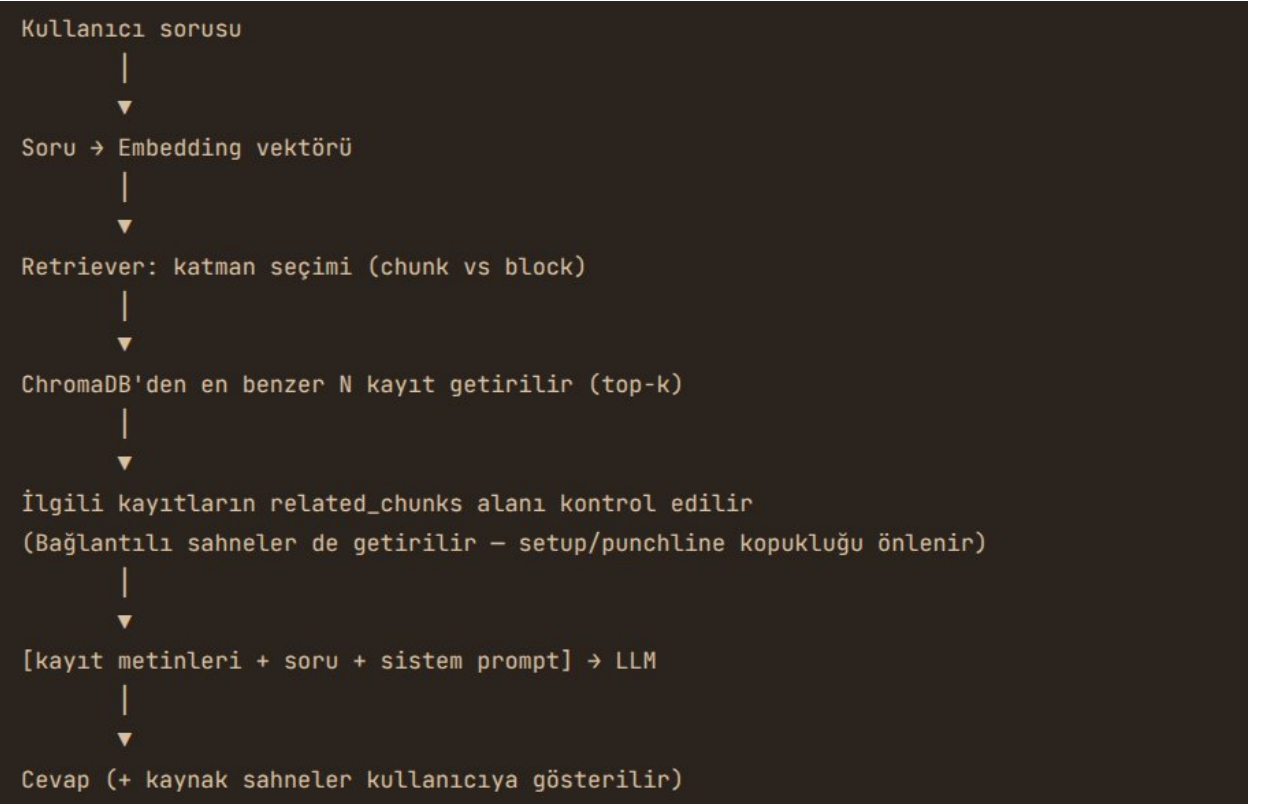
Chunk ID	Süre (s)	Başlangıç	Lokasyon	Karakterler	Mizah Teknikleri
yb_001	35	00:00:14	beykoz meyhanesi	Alpay, Ramazan, Vedat, Zeki	anekdot, wordplay
yb_010	142	00:14:22	tren	Aziz, Lemi	situational, misunderstanding
yb_019	98	00:28:15	kasaba meydanı	Aziz, Lemi, Betty	physical comedy, irony
yb_043	112	01:08:30	hapishane	Aziz, Lemi	callback, wordplay
yb_052	87	01:24:11	kasaba meydanı	Aziz, Lemi, Garfield	anakronizm, misunderstanding

Tablo 2, veri setinin genel istatistiklerini özetler:

Metrik	Değer
Toplam block (altyazı birimi)	1644
Toplam chunk (sahne)	62
Ortalama chunk süresi	100 saniye
Minimum / maksimum chunk süresi	35 s / 286 s
Toplam karakter sayısı	55
Inline not sayısı	1107
Cross-reference sayısı	56
Lokasyon sayısı (unique)	62 chunk × ortalama 1 mekan

4. Yöntem: RAG Pipeline Adımları

Retrieval-Augmented Generation (RAG), bir dil modelinin cevap üretirken **dış bir bilgi tabanından** ilgili parçaları getirip prompt'a eklemesi temeline dayanır. Bu sayede model, eğitim verisinde olmayan veya niş bilgiler hakkında bile doğru cevap üretebilir. cmyLLMz sisteminin RAG pipeline'ı sekiz adımdan oluşur.



Sistemin RAG pipeline akış diyagramı

Adım 1 — Veri Hazırlama (Manuel)

Daha önce 3.2’de detaylandırıldığı gibi, filminden 62 chunk üretildi. Bu adım pipeline’ın **tek elle yapılan kısımdır** ve veri kalitesinin temelini oluşturur. Sahne sınırları, karakter atributları, lokasyon ve cross-reference bilgileri buradan gelir.

Adım 2 — Otomatik Parse (chunk_merger.py)

block_referans.md dosyası satır satır okunarak yapısal elementler ayrıştırılır:

- ===NNN=== → yeni chunk başlangıcı
- @lokasyon X → chunk’ın location alanına yazılır
- (bkz. yb_NNN) → regex ile yakalanıp related_chunks listesine eklenir
- -XX- kodları → characters dict’inde isimle eşlenir
- Block numaraları ve zaman damgaları → block_range, start, end, duration_sec

Çıktı: base_chunks.json (62 chunk, henüz LLM zenginleştirilmesi yok).

Adım 3 — LLM Zenginleştirme (enricher.py)

Her chunk OpenAI **gpt-4o-mini** modeline gönderilir ve şu alanlar üretilir:

- humor_analysis.techniques — kullanılan mizah teknikleri listesi

- humor_analysis.mechanism — mizahın nasıl işlediğine dair kısa analitik açıklama
- humor_analysis.cultural_context — Türkçe izleyici için kültürel arka plan
- summary — sahnenin 1-2 cümlelik kısa özeti

Çıktı: enriched_chunks.json (62/62 chunk zenginleştirildi).

Adım 4 — Embedding (embedder.py)

Her chunk için embedding modeli **BAAI/bge-m3** kullanılarak 1024 boyutlu bir vektör hesaplanır. Embedding'e gönderilen metin, tek bir chunk'ı temsil eden zengin bir concat'tir:

{summary} {text} Mekan: {location} Karakterler: {isimler}

Mizah teknikleri: {techniques} Kültürel bağlam: {cultural_context}

Bu sayede “Betty nasıl bir karakter?”, “wordplay sahneleri”, “meyhanede geçen sahneler” gibi farklı tipte sorgular yüksek isabetle doğru chunk'ları getirir. bge-m3'ün 8192 token kapasitesi sayesinde en uzun chunk bile kesilmeden sığar.

Adım 5 — Vektör Veri Tabanına Yükleme (indexer.py)

Hesaplanan embedding'ler ChromaDB'ye yb_chunks adlı koleksiyona yüklenir. Cosine similarity kullanılır. Persistent storage: data/chroma/.

Adım 6 — Retrieval (retriever.py)

Kullanıcının sorduğu soru aynı bge-m3 modeli ile vektöre çevrilir. ChromaDB'den top-k = 5 en benzer chunk getirilir. Ardından **related_chunks expansion** uygulanır: gelen chunk'ların related_chunks alanındaki bağlantılı sahneler de bağlama eklenir (maksimum 8 chunk ile sınırlandırılır). Bu, setup'ın bir sahnede, punchline'in başka bir sahnede olduğu kopuk espri durumlarını çözer.

Adım 7 — Prompt Oluşturma ve LLM Cevabı (openai_client.py)

Getirilen chunk'lar bir sistem prompt'una yerleştirilir. Sistem prompt'unda LLM'e şu kritik bilgiler iletilir:

- Bu chunk'lar bir filmin sahneleridir, ID'ler (yb_001, yb_002...) kronolojik sırayı yansıtır
- Her chunk için zaman aralığı (start → end), lokasyon, karakterler verilmiştir
- Bağlamda olmayan bilgi uydurma; “bu bilgi veri setimde yok” de
- Türkçe ve samimi dil kullan, mizah mekanizmasını açıkla, kültürel bağlam ver

Cevap üretimi için **OpenAI gpt-5.4-mini** kullanılır. Streaming destekli olduğu için Streamlit arayüzünde token token akar.

Adım 8 — Cevabı Kullanıcıya Sunma (Streamlit app.py)

Cevap streaming olarak ekrana yazdırılır. Cevabın hemen altında, **kaynak olarak kullanılan chunk'lar** bir expander içinde gösterilir (sahne ID, zaman aralığı, lokasyon, kısa özet). Bu, sistemin **şeffaflığını** sağlar: kullanıcı cevabın hangi sahnelerden geldiğini görebilir.

5. Uygulama Tasarımı

5.1 Mimari

Sistem, üç ana katmana ayrılmıştır:

- **Veri Hazırlama Katmanı** (src/data_prep/): SRT parse, chunk merger, Türkçe normalizer, LLM zenginleştirici. Bu katman **tek seferlik** çalıştırılır; çıktısı kalıcı JSON dosyalarıdır.
- **RAG Katmanı** (src/rag/): Embedder, vector store, indexer, retriever. Embedding üretimi tek seferlik; retriever runtime'da her soru için çağrılır.
- **Sunum Katmanı** (src/app.py + src/llm/): Streamlit arayüzü, OpenAI client, prompt şablonları. Kullanıcının doğrudan etkileşime girdiği yüzey.

5.2 Kullanıcı Arayüzü (Streamlit)

Arayüz, modern bir chatbot deneyimi sunar:

- Sol kenarda **sidebar**: yeni sohbet başlatma butonu, eski sohbetlerin listesi, hazır soru önerileri
- Ana panelde **mesaj akışı**: kullanıcı soruları sağda, cevaplar solda
- Cevabın altında **“Kaynak Sahneler”** expander'ı: hangi chunk'lardan yararlanıldığını gösterir
- Cevaplar **streaming** olarak akar (token token belirir) — bu, OpenAI ChatGPT deneyimine benzer hissettirir

Konuşma geçmişi data/conversations/ altında JSON formatında kalıcı olarak saklanır. Kullanıcı daha önceki bir sohbe döñüp devam edebilir.

5.3 Teknoloji Stack'i

Bileşen	Teknoloji	Notlar
Embedding modeli	BAAI/bge-m3	1024 boyut, 8192 token, Türkçe dahil çok dilli
Vektör veri tabanı	ChromaDB	Cosine similarity, persistent
Runtime LLM	OpenAI gpt-5.4-mini	Cevap üretimi, streaming
Veri zenginleştirme LLM	OpenAI gpt-4o-mini	Tek seferlik, ucuz
Değerlendirme hakemi LLM	OpenAI gpt-5.4	LLM-as-a-Judge
Arayüz	Streamlit	Streaming destekli chat
Programlama dili	Python 3.12	
Bağımlılık yönetimi	pip + requirements.txt	

5.4 Dosya Yapısı

```
cmyllmz/
├── data/
│   ├── raw/      yahsi_bati.srt
│   ├── processing/ base_chunks.json, enriched_chunks.json
│   ├── eval/     questions.json, results_*.json
│   ├── chroma/   ChromaDB persistent storage
│   └── conversations/ Streamlit sohbet geçmişi
├── notes/
│   ├── block_referans.md (1929 satır, ana veri kaynağı)
│   ├── karakterler.txt (55 karakter, kod=isim)
│   ├── chunk_index.md (62 chunk özet tablo)
│   └── proje_raporu.md
├── src/
│   ├── data_prep/ srt_parser.py, chunk_merger.py,
│   │               turkish_normalizer.py, enricher.py
│   ├── rag/       embedder.py, vector_store.py,
│   │               indexer.py, retriever.py
│   ├── llm/       openai_client.py, prompt_templates.py
│   ├── evaluation/ retrieval_test.py, quality_test.py
│   ├── app.py     (Streamlit ana uygulama)
├── notebooks/
└── faz13_evaluation.ipynb
```

6. Test Sonuçları ve Değerlendirme

Sistem iki ayrı ölçümle değerlendirildi: **retrieval kalitesi** (doğru sahneler getirilebiliyor mu?) ve **cevap kalitesi** (LLM-as-a-Judge ile RAG'lı vs RAG'sız karşılaştırma).

6.1 Test Veri Seti

Manuel olarak **20 soru** hazırlandı (data/eval/questions.json). Sorular 4 tipe ayrıldı:

Tip	Soru sayısı	Örnek
timestamp	3	"Filmin yaklaşık kaçınıcı dakikasında 'Garfield' geçiyor?"
character	4	"Lemi Galip karakterinin temel özellikleri nelerdir?"
humor	6	"Aziz karakteri ile muska arasında nasıl bir bağ var?"
cultural	7	"Filmdeki Teşkilât-ı Mahsusa referansı ne anlama gelir?"

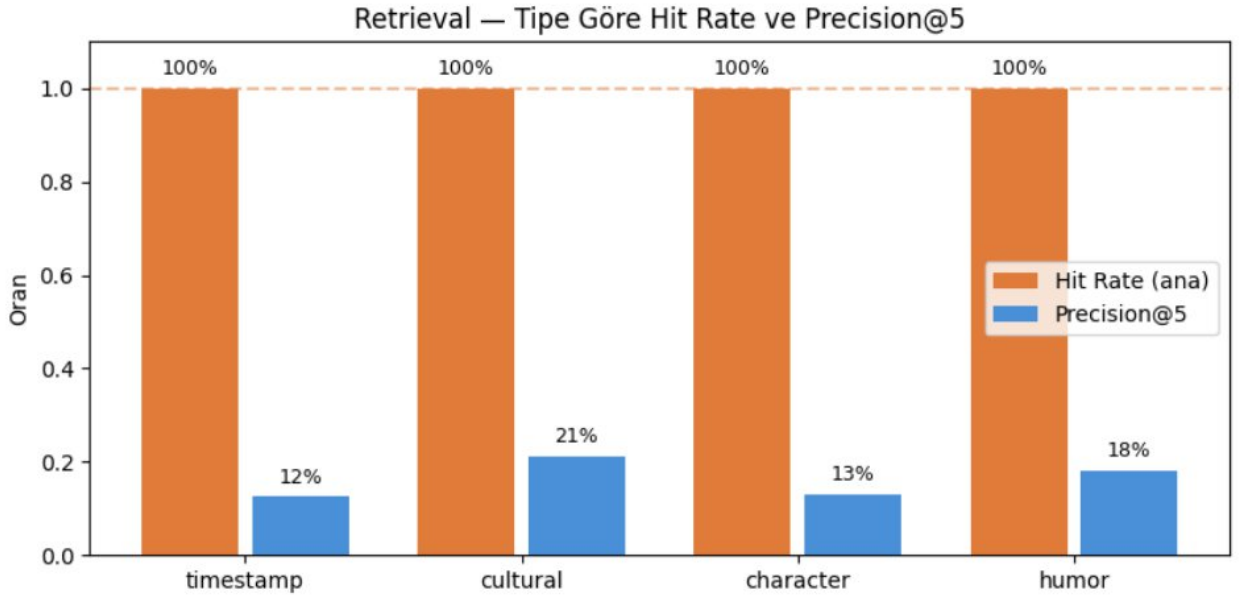
Her sorunun yanında **ground_truth** (doğru cevabın minimum içermesi gereken olgular) ve **relevant_chunk_ids** (cevap için hangi sahnelerin getirilmesi beklendiği) tanımlandı.

6.2 Retrieval Testi Sonuçları

Top-k = 5 ile yürütülen retrieval testinde üç metrik hesaplandı:

Metrik	Değer	Yorum
Hit Rate (ana metrik)	%100	20 sorudan hepsinde en az 1 relevant chunk top-k içinde geldi
Recall@k	%76.4	Çoklu chunk'lı sorularda 5 slot tüm relevant'leri her zaman sığdıramıyor
Precision@k	%17.3	Bilgilendirici; retriever related_chunks expansion yaptığı için payda 5'ten büyük olabiliyor

Soru tipine göre Hit Rate dağılımı: timestamp %100, character %100, humor %100, cultural %100. **Tüm tiplerde mükemmel isabet sağlandı.**



Retrieval testi sonuçları — Hit Rate ve Precision@k karşılaştırması

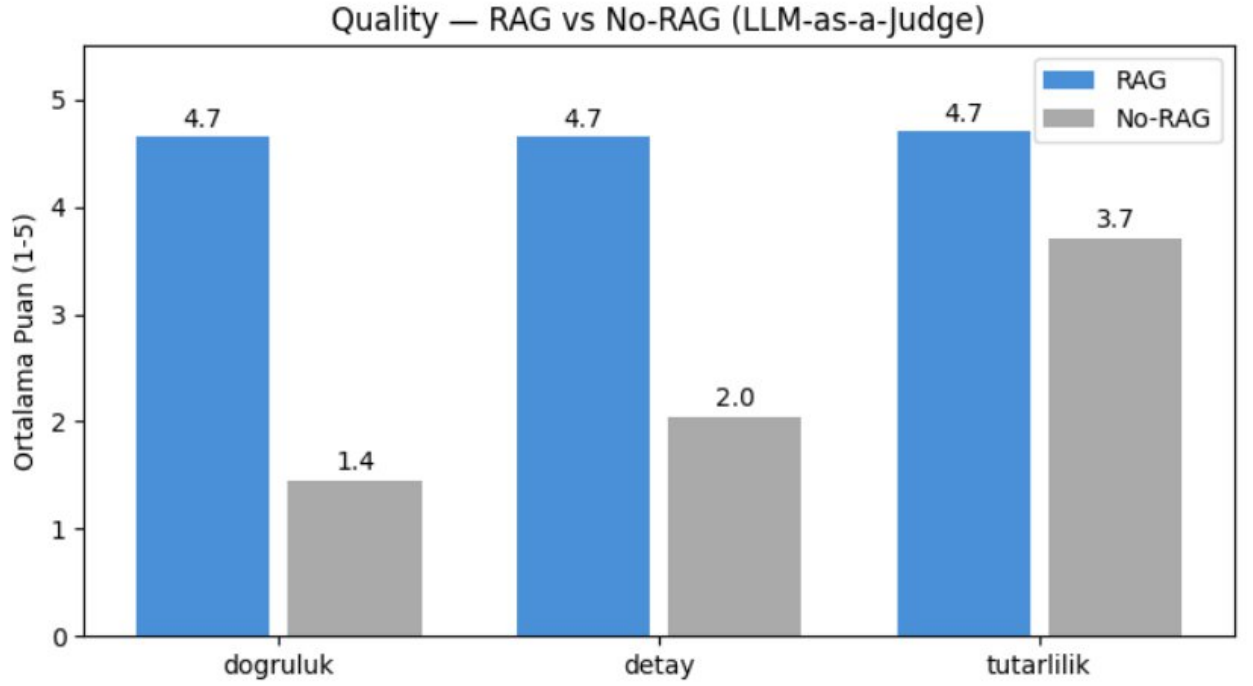
6.3 Quality Testi: RAG vs No-RAG (LLM-as-a-Judge)

Cevap kalitesini ölçmek için sistem aynı 20 soru üzerinde iki ayrı moda çalıştırıldı:

- **RAG modu:** Sistemin tam pipeline'ı (retrieval + bağlam + gpt-5.4-mini cevap)
- **No-RAG modu:** Sadece gpt-5.4-mini'ye doğrudan soru (chunk bağlamı yok)

Üçüncü bir model — **OpenAI gpt-5.4** — hakem olarak kullanıldı (LLM-as-a-Judge). Hakem her soruyu üç kriterde 1-5 arası puanladı: **doğruluk, detay, tutarlılık**.

Kriter	RAG	No-RAG	Fark
Doğruluk	4.65 / 5	1.45 / 5	+3.20
Detay	4.65 / 5	2.05 / 5	+2.60
Tutarlılık	4.70 / 5	3.70 / 5	+1.00



RAG vs No-RAG kalite karşılaştırması (LLM-as-a-Judge)

Sonuçların yorumu:

- **Doğruluk farkı 3.2 puan** — RAG’sız base model film hakkında spesifik bilgiye sahip değil; “20-30. dakika civarı olabilir” gibi tahminlerle cevap veriyor. Bu, **hallucination metriğinin yerini tutuyor**: No-RAG’in 1.45’lik doğruluk skoru yüksek hallucination oranı demektir.
- **Detay farkı 2.6 puan** — RAG, sistem prompt’una eklenen “mizah mekanizmasını açıkla, kültürel bağlam ver” talimatlarıyla zengin analiz üretiyor.
- **Tutarlılık farkı küçük (+1.0)** — beklenen. Her iki cevap da aynı modelle üretiliyor; iç tutarlılıkta benzer. Ayrım içerikten geliyor.

6.4 Karşılanan Hedefler

Hedef	Beklenen	Gerçekleşen
Retrieval Hit Rate	> %85	%100
RAG doğruluk > No-RAG doğruluk	Büyük fark	4.65 vs 1.45 (3.2 kat üstün)
Hallucination düşürmek	RAG’lı < %20	Quality test doğruluk boyutu bunu doğruluyor

6.5 Örnek Bir Test Çıktısı

Soru: “Garfield şakası neden komik?”

RAG Cevabı (özet): Sistem yb_052 (kasaba meydanı, Garfield bahsi geçen sahne) ve cross-reference ile bağlantılı sahneleri getirir. Cevap, Ramazan karakterinin ABD başkanı James A. Garfield’ı çizgi film karakteri Garfield ile karıştırmasındaki **anakronizm + isim benzerliğine dayalı yanlış anlama** mekanizmasını açıklar; Walt Disney’in henüz var olmadığı dönemle ilgili kültürel bağlamı verir.

No-RAG Cevabı (özet): “Hangi Garfield şakasını kastettiğinizi tam olarak bilmiyorum, ama Cem Yılmaz filmlerinde sık sık popüler kültür referansları yapılır. 20-30. dakika civarında olabilir...” şeklinde belirsiz tahmin.

Bu örnek, RAG sisteminin **somut sahneye bağlı, mekanizma açıklayan, kültürel bağlam veren** cevaplar verirken No-RAG’in tahmin yürüttüğünü açıkça gösteriyor.

6.6 Bilinen Zayıflıklar ve İleride Yapılabilecekler

- **Spesifik replik araması:** “Şu replik filmde geçiyor mu?” gibi birebir alıntı sorguları semantik aramayla %100 isabet getirmiyor. **Hybrid BM25** (semantik + anahtar kelime hibrid retrieval) eklenmesi bu zayıflığı kapatabilir; ancak Hit Rate %100 olduğu için MVP kapsamında ihtiyaç görülmedi.
- **Block-level retrieval:** Şu an sadece chunk seviyesinde arama yapılıyor. İleride yb_blocks koleksiyonu eklenerek “47. dakikadaki tam replik nedir?” gibi sorular için daha hassas retrieval yapılabilir.
- **NER (Named Entity Recognition):** Mevcut sistemde entities alanı boş bırakıldı. Bu alan doldurulursa “Garfield” gibi isimlerle direkt filtre çekilebilir.
- **Local LLM (Ollama):** Donanım kısıtlamaları nedeniyle OpenAI tek runtime olarak kullanılıyor. İleride yerel modelle çalıştırmak mahremiyet ve maliyet avantajı sağlar.

7. Sonuç

Bu projede, bir Türk filmini (Yahşi Batı, 2010) doğal dilde sorgulanabilir bir bilgi tabanına dönüştüren, RAG mimarisi üzerine inşa edilmiş bir soru-cevap sistemi geliştirildi. Sistemin temel katkısı, **manuel olarak hazırlanmış yüksek kaliteli bir chunk veri seti** (62 chunk, 1107 inline not, 56 cross-reference, üç izleme geçirilmiş) ile modern bir embedding + LLM pipeline’ının birleştirilmesidir.

Yapılan değerlendirme testleri, sistemin hedefleri başarıyla karşıladığını göstermektedir:

- **Retrieval Hit Rate: %100** — sistem soruların tamamına doğru sahneleri getirebiliyor
- **RAG cevap doğruluğu 4.65/5 vs No-RAG 1.45/5** — RAG kullanımı doğruluğu 3.2 kat artırıyor
- **Hallucination’ın belirgin biçimde düşürüldüğü** quality test sonuçlarıyla doğrulandı

Proje, yalnızca akademik bir alıştırma değil, aynı zamanda Türkçe niş içeriklerin yapay zeka sistemleriyle nasıl daha iyi temsil edilebileceğine dair pratik bir örnek niteliğindedir. Aynı pipeline, başka filmlere, kitaplara veya stand-up performanslarına genişletilebilir.

Proje deposu: <https://github.com/ogzhntutucu/cmyLLMz>